

Problem Set 2: Predicting Poverty “Wars of nations are fought to change maps. But wars of poverty are fought to map change” M. Ali

Andrés Rueda López - a.ruedal@uniandes.edu.co

October 21, 2024

Todos los documentos y archivos utilizados para la solución del problema se encuentran en el siguiente repositorio: [Repositorio en GitHub](#). Adicionalmente, se ha creado la sección de Apéndice 5 para mayor detalle sin incurrir en exceso de paginas.

1 Introducción

La medición de la pobreza es una tarea compleja y costosa que impacta directamente en la creación e implementación de políticas públicas eficaces. En 2020, el DANE estimó que cerca del 20% de la población colombiana vivía en situación de pobreza [3]. Sin embargo, estas mediciones tradicionales pueden ser insuficientes, lo que dificulta la toma de decisiones orientadas a reducir este índice de manera sostenida.

En los últimos años, con el auge de la disponibilidad de datos, se ha explorado el uso de técnicas de *Machine Learning* para mejorar la precisión en la predicción de la pobreza. El Banco Mundial destaca que estas técnicas pueden proporcionar herramientas estratégicas para los gobiernos, siempre que se cuente con datos de calidad [5]. Ejemplos exitosos incluyen estudios en Costa Rica, donde modelos como *Random Forest* y *Gradient Boosted Trees* demostraron ser eficaces, con este último alcanzando un puntaje *F1* del 68.4%, y resaltando la educación como un predictor relevante [4]. Otros enfoques han combinado datos de encuestas con imágenes satelitales, logrando identificar patrones complejos a través del uso de información geoespacial [2]. Además, una revisión reciente sobre inteligencia artificial en este campo sugiere que técnicas como *XGBoost* superan a los modelos lineales tradicionales, especialmente al incorporar variables geográficas y socioeconómicas [1].

Este trabajo busca predecir el estado de pobreza de los hogares encuestados por el DANE en el “*Empalme de las Series de Empleo, Pobreza y Desigualdad - MESE*”. La encuesta proporciona una base de datos rica en características socioeconómicas tanto de los hogares como de sus miembros. Se emplean modelos de clasificación como *Random Forest*, Regresión Logística y Redes Neuronales Profundas, optimizando y regularizando los datos de entrada para maximizar el desempeño medido a través del puntaje *F1* en el conjunto de validación.

Un aspecto clave del proceso es la creación y selección de variables predictoras que mejoren la precisión de las predicciones. Los mejores resultados se obtuvieron con modelos de baja y alta complejidad: regresión logística y redes neuronales profundas, respectivamente. Esto resalta que tanto un tratamiento adecuado de los datos como la construcción rigurosa de variables son cruciales para el éxito del modelo. Además, los resultados sugieren que existen relaciones complejas entre las variables que modelos avanzados, como las redes neuronales, logran capturar de manera efectiva.

2 Datos

Para obtener los datos necesarios se utilizó información provenientes del DANE y de la misión “Uniformidad de la Serie Empleo, Pobreza y Desigualdad – MESE”. Teniendo en cuenta las reglas estipuladas por la competencia en su sitio oficial, únicamente se utilizaron los datos proporcionados por la misma. Estos datos consistían en 5 archivos en formato .csv y un documento en PDF. El apéndice 5 contiene una breve descripción de ellos.

Con esto se procedió a realizar un análisis exploratorio más profundo con el fin de construir un conjunto de entrenamiento y validación adecuados, que permitieran obtener las mejores predicciones posibles. A continuación se describe de manera detallada, el proceso de selección y construcción de variables utilizado dentro del conjunto de entrenamiento y validación.

2.1 Filtrado Inicial (`train_hogares`, `test_hogares`)

En primer lugar, se examinó el conjunto de datos de entrenamiento (`train_hogares`) y de validación (`test_hogares`) correspondientes a los hogares, considerando que la predicción de pobreza se realiza a nivel del hogar. El conjunto de entrenamiento se convirtió en un DataFrame de dimensiones (164,960 x 23) y el de validación en un DataFrame de dimensiones (66,168 x 16). Dado que las dimensiones de ambos conjuntos no coincidían, fue necesario eliminar aquellas variables en el conjunto de entrenamiento que no estaban presentes en el conjunto de validación.

Después de realizar el primer ajuste, se examinó el conjunto de entrenamiento para analizar el comportamiento de la variable objetivo, con el fin de identificar su distribución y anticipar posibles tratamientos ante un posible desbalance.

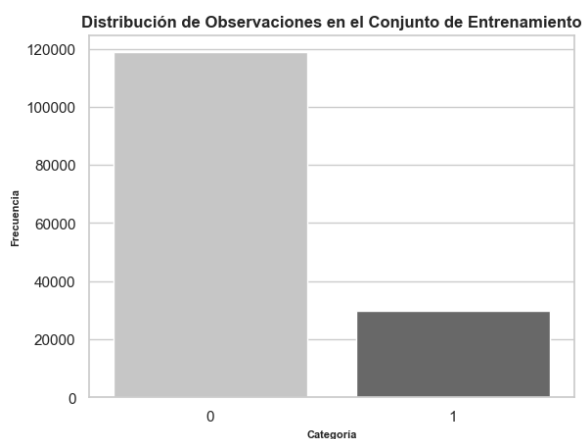


Figure 1: Distribución de la variable objetivo (1 indica que el hogar es pobre y 0 indica que no lo es)

En la figura 6, se observa claramente una superioridad en el número de observaciones que indican que el hogar no es pobre, en comparación con aquellas que señalan lo contrario. Esto evidencia un **desbalance de clases moderado**, donde la clase minoritaria es cuatro veces menor que la mayoritaria. En secciones posteriores, se abordarán los desafíos que este desbalance presenta y se detallarán las estrategias que se implementarán para evitar que el modelo genere predicciones erróneas al favorecer la clase mayoritaria.

2.2 Filtrado Inicial (`train_personas`, `test_personas`)

El procedimiento de filtrado inicial para las bases de datos de las personas fue bastante similar al realizado previamente en el conjunto de hogares. Se procedió a convertir el conjunto de entrenamiento (`train_personas`) en un DataFrame de dimensiones (543109 x 135) y el conjunto de validación (`test_personas`) en un DataFrame de dimensiones (219644 x 63). Nuevamente, la cantidad de variables presentes en entrenamiento es superior a las que existen en validación por lo que se procedió a remover aquellas variables del conjunto de entrenamiento que no estuviesen presentes en el conjunto de validación.

2.3 Selección y creación de variables explicativas

Aunque la predicción se realiza a nivel del hogar, es fundamental destacar que la base de datos de personas contiene mucha más información (en términos de variables) que puede ser útil para determinar si un hogar es pobre o no. Por este motivo, fue necesario vincular a las personas con sus hogares, lo cual se logró mediante el `id` que identifica a cada hogar.

Con este punto claro, se procedió a construir un conjunto de datos para los hogares (tanto de entrenamiento como de evaluación) que incluyera únicamente las variables necesarias para ajustar los diferentes modelos. Este proceso implicó seleccionar algunas variables de las bases de datos, transformarlas en ciertos casos y, a partir de ellas, crear nuevas variables que permitan mejorar el rendimiento del modelo.

A continuación, se presentan brevemente las variables más relevantes que se utilizarán en los modelos introducidos en las secciones posteriores. La creación detallada de estas variables, junto con otras de menor interpretabilidad que también fueron utilizadas, se explica en el Apéndice 5.

1. **Hacinación:** Se definió una medida de hacinación h_i para cada hogar i a partir del número de personas que habitan en el hogar y el número de cuartos que son usados en el hogar para dormir.
2. **Escolaridad media:** Se definió un promedio de escolaridad $\bar{e}_i$ para cada hogar i como la suma ponderada del grado de escolaridad más alto alcanzado por cada miembro j del hogar
3. **Subsidios:** Teniendo en cuenta que existen múltiples subsidios dentro de la base de datos, se utilizaron los que pueden tener un mayor indicio de precariedad económica por parte del hogar.
4. **Cantidad de personas ocupadas en el hogar:** Entendiendo la ocupación como un parámetro binario asignado a cada persona como: $o_j = 1$ si la persona j esta ocupada, se calculo el número de personas ocupadas en el hogar O_i como un indicativo de desempleo.
5. **Número de personas cotizando pensión en el hogar:** Entendiendo la cotización en un fondo pensiones como un parámetro binario asignado a cada persona como: $c_j = 1$ si la persona j cotiza , se calculo el número de personas que se encuentran realizando cotizaciones en algún fondo de pensiones dentro del hogar C_i como un indicativo de formalidad laboral.
6. **Titulo del hogar:** Para cada hogar, se determinó en base a la encuesta que tipo de vivienda es dentro de las siguientes opciones: (a. Propia, totalmente pagada b. Propia, la están pagando c. En arriendo o subarriendo d. En usufructo e. Posesión sin titulo (ocupante f. Otra). Esta variable fue construida como una **variable categórica**.
7. **Arriendo estimado del hogar:** Se tuvo en cuenta la estimación del arriendo del hogar por parte de los propietarios/ habitantes. Dado que en múltiples ocasiones este valor no era reportado dentro de la base de datos, en caso de que el valor fuera nulo se utilizo la media del resto de observaciones como estimador de estos valores de ser necesario.
8. **Número de personas en el hogar:** Adicional a la medida de hacinamiento previamente definida, se tuvo en cuenta le número de personas n_i que habita en el hogar i como otra medida de hacinamiento que puede brindar información sobre precariedad del hogar.

Finalmente, un análisis estadístico más formal se realizó buscando el comportamiento de las correlaciones y análisis descriptivo de las variables. Esto se hizo con el fin de analizar posibles ajustes o estandarizaciones necesarias en los predictores.

Table 1: Estadísticas descriptivas de algunas variables seleccionadas

Medida	P6210	P6020	P6040	P6050	P6090	mean_escolaridad	Subsidios
count	164960.00	164960.00	164960.00	164960.00	164960.00	164960.00	164960.00
mean	4.31	1.53	37.44	2.27	1.07	5.13	1.26
std	1.05	0.28	16.88	0.88	0.23	1.09	1.54
min	1.00	1.00	5.67	1.00	1.00	1.00	0.00
25%	3.65	1.33	24.00	2.00	1.00	5.00	0.00
50%	4.33	1.50	33.50	2.25	1.00	5.00	1.00
75%	5.00	1.67	48.20	2.75	1.02	6.00	2.00
max	9.00	2.00	102.00	8.38	9.00	9.00	17.00

Note que la Tabla 1 muestra que los predictores han sido estandarizados, esto se realizó debido a que las variables seleccionadas poseían unidades de medida muy diferentes lo que podría causar

problemas en la estimación. Aunque no todos los predictores se muestran en 1, este análisis se realizó con todas las variables para conocer y entender los datos.

Adicionalmente, se realizó un análisis de covarianza entre las variables para conocer si existía información redundante dentro de las variables.

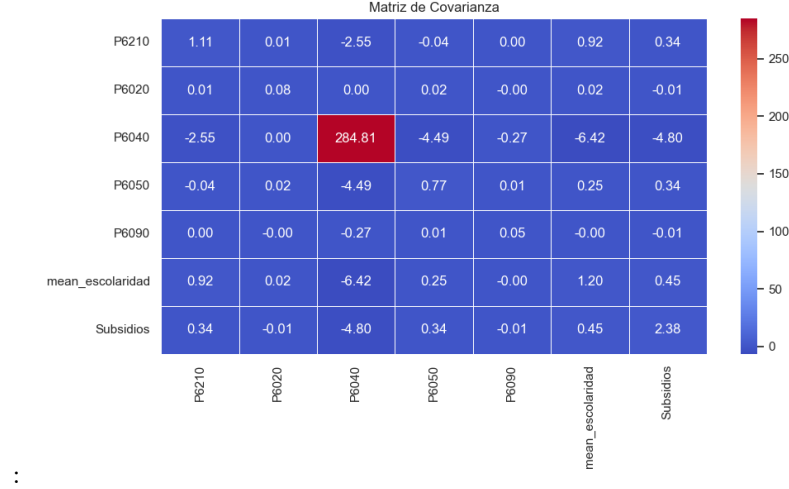


Figure 2: Matriz de Varianza-Covarianza de algunos predictores

En este caso, se busca que en la Figura 2 no existan valores altos de covarianza entre las variables dado que esto podría indicar correlación o información redundante. Nuevamente vale la pena mencionar que este análisis se realizó con todos los predictores.

3 Modelos y Resultados

Para todos los modelos presentados a continuación, se asume una matriz de predictores X de entrenamiento, compuesta por todas las variables previamente explicadas. Cada observación $x_i = (x_1, x_2, \dots, x_m) \in X$ está asociada con su respectiva etiqueta Y_i .

Dado que es fundamental evaluar el rendimiento de los modelos estimados, se ha decidido dividir el conjunto de datos de entrenamiento en dos partes: un conjunto de entrenamiento puro (90% de la muestra) y un conjunto de prueba (10% de la muestra). Esta división permite evaluar el desempeño de los modelos con datos no vistos previamente. Esta división únicamente se realizó con el fin de evaluar los modelos sin necesidad de usar los submissions limitados de la competencia.

3.1 Métrica de evaluación

Los modelos que a continuación se presentan fueron entrenados y evaluados para maximizar el puntaje del **F1-Score**. Recuerde que el F1-Score viene dado por:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

Para cada modelo se espera mostrar el puntaje F1-Score obtenido en evaluación (Kaggle), junto con una breve descripción de su funcionamiento y los parámetros utilizados.

3.2 Método de remuestreo (SMOTE)

Teniendo en cuenta que previamente se había visto un desbalanceo moderado (la clase minoritaria es 4 veces inferior a la clase mayoritaria) en los datos, se decidió optar por la estrategia conocida como Synthetic Minority Over-sampling Technique (SMOTE).

En este caso, se busca generar muestras sintéticas de la clase minoritaria hasta alcanzar una proporción deseada utilizando métodos basados en agrupamientos. Para ello, se crea un dato sintético

a partir de los k vecinos más cercanos (siendo k un parámetro que se debe definir) mediante el valor medio o una ponderación lineal de esos vecinos. Tras realizar una búsqueda exhaustiva con una rejilla de hiperparámetros (en este caso se usó el método de GridSearch de Python en sklearn), se determinó que el valor óptimo es $k = 7$ vecinos, y que la clase minoritaria debería representar el 75% del tamaño de la clase mayoritaria (en 5 se encuentra una ilustración del proceso).

3.3 Modelos de Clasificación

A continuación, se presentan todos los modelos de clasificación utilizados. Recuerde que un modelo de clasificación en este caso es de la forma:

$$Poor = I(Inc > Pl) \quad (2)$$

Donde I es una función indicadora que toma 1 si el ingreso de la familia es inferior a cierto umbral de pobreza.

3.3.1 Regresión Logística

El modelo de regresión utilizado sigue la **función logística**:

$$\hat{Y} = \frac{e^{X\beta}}{1 + e^{X\beta}}$$

donde el objetivo es estimar los coeficientes β que maximizan el **F1-Score**. Este modelo se ajusta mediante *máxima verosimilitud*, utilizando métodos numéricos debido a su **naturaleza no lineal**, lo que impide una solución analítica directa.

El entrenamiento se realizó utilizando una **búsqueda en grilla** proporcionada por el paquete **sklearn** de Python. Los parámetros óptimos encontrados fueron $c = 100$ y `solver = liblinear`, obteniendo un score en Kaggle de $F1 = 0.638$

3.3.2 Random Forest

El modelo **Random Forest** es un conjunto de árboles de decisión, donde cada árbol se entrena utilizando un subconjunto aleatorio del conjunto de datos y de sus características. Las predicciones se obtienen mediante una votación de la mayoría entre los árboles del bosque, lo que reduce la varianza del modelo y mejora su capacidad de generalización.

En este experimento, se encontró como configuración óptima una profundidad máxima de 10 (`max_depth`), un mínimo de 30 muestras para dividir nodos (`min_samples_split`), y 500 árboles (`n_estimators`). El modelo alcanzó un score en Kaggle de $F1 = 0.576$ en el conjunto de prueba, con un umbral de decisión óptimo de 0.30.

3.3.3 CARTS (Árboles de Decisión)

El modelo **CARTS (Classification and Regression Trees)** construye un árbol de decisión dividiendo recursivamente los datos en función de las características que mejor separan las clases objetivo. Es un modelo fácil de interpretar, aunque propenso al sobreajuste si no se controla adecuadamente.

Para este experimento, se utilizó la clase `DecisionTreeClassifier` de **sklearn** con un. El modelo fue entrenado utilizando un conjunto de datos balanceado. En la evaluación final, se obtuvo EN Kaggle un $F1 = 0.469$.

3.3.4 XGBoost

El modelo **XGBoost (Extreme Gradient Boosting)** es un algoritmo basado en boosting de árboles de decisión. En este enfoque, cada árbol intenta corregir los errores cometidos por los árboles anteriores, permitiendo capturar relaciones complejas en los datos. Además, XGBoost incorpora regularización, lo que ayuda a evitar el sobreajuste del modelo.

Durante la búsqueda de hiperparámetros, se encontró como configuración óptima un número de 100 estimadores (`n_estimators`). El mejor **F1-Score** alcanzado fue de 0.605, con un umbral de decisión óptimo de 0.30. Se utilizó la opción `use_label_encoder=False` para evitar advertencias durante la ejecución.

3.4 Redes Neuronales Profundas (Best Model)

3.4.1 Funcionamiento

El modelo que se presenta a continuación fue el que obtuvo los mejores resultados en la competencia de Kaggle. Este enfoque se basa en redes neuronales profundas (DNN, por sus siglas en inglés), un método de aprendizaje automático inspirado en el funcionamiento del sistema neuronal humano. La red está compuesta por múltiples capas (*layers*), cada una de ellas integrada por neuronas (*neurons*) interconectadas de forma densa y completamente conectada. El objetivo de este diseño es capturar de manera eficiente las relaciones complejas entre las variables (*features*) del modelo, permitiendo mejorar la capacidad predictiva.

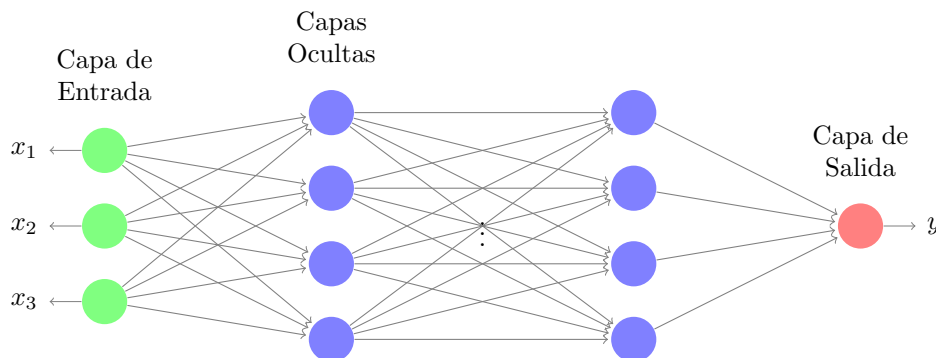


Figure 3: Esquema de una red neuronal profunda con múltiples capas ocultas.

En primer lugar, la red propaga los datos a través de una capa de entrada (*input layer*), donde cada nodo o neurona representa una variable de la matriz de las X . Posteriormente, la red se propaga a través de las capas intermedias (*hidden layers*) las cuales realizan transformaciones lineales sobre los datos, esto es que para cada neurona se realiza una estimación de la forma:

$$z = \sum_{i=1}^n w_i x_i + b \quad (3)$$

Donde w_i representa el peso (*weight*) otorgado a la neurona y b representa el termino de sesgo (*bias*). Una vez se realiza la ponderación, la neurona aplica una función no lineal sobre la estimación con el fin de evitar que el modelo se convierta en la combinación lineal de estimaciones lineales, la cual se reduciría a una única estimación lineal. Este proceso se repite en cada neurona y en cada capa oculta, lo cual se conoce como *forward propagation*.

Finalmente, los datos llegan a una capa de salida (*output layer*), donde se realiza una última ponderación y se genera la predicción del valor de la variable de interés en función de sus predictores. A partir del valor de salida obtenido y el valor real asociado al dato (en el contexto de aprendizaje supervisado), se ajustan los parámetros del modelo mediante el método de retropropagación (*back-propagation*).

Este método consiste en calcular las derivadas parciales de los parámetros del modelo con respecto a la función de pérdida. Sin embargo, dado que estas derivadas se encadenan a través de múltiples capas, el proceso puede volverse computacionalmente complejo. Para hacer el ajuste eficiente, la retropropagación utiliza algoritmos avanzados de descenso del gradiente, los cuales permiten encontrar los parámetros óptimos que minimizan la función de pérdida.

3.4.2 Aplicación a caso de estudio

Como se menciono previamente, este modelo al igual que los presentados previamente utilizó las variables mencionadas en la sección 2. Sin embargo, para mayor claridad, a continuación se muestra el listado total de variables utilizado.

Aquellas con mayor relevancia en términos de interpretabilidad se han ya explicado en 2, sin embargo, las variables restantes únicamente se mencionan en esta sección para posible replicabilidad.

Por otro lado, este modelo recibió como entrada una matriz de X y el vector Y que ha sido primeramente tratado con el método de re-muestreo (SMOTE) previamente explicado en 3.2. Los parámetros allí explicados fueron los que mejor desempeño mostraron en el desempeño del modelo.

3.4.3 Arquitectura de la Red

Teniendo en cuenta que la arquitectura de la red, constituye un componente fundamental en la estimación del modelo, se utilizó una gran cantidad de combinaciones y arquitecturas que permitieran abordar diferentes aproximaciones al problema. Para ello, se construyó una grilla manual que evaluaba diferentes arquitecturas comunes en la literatura, y se realizó su submission en Kaggle para evaluar su desempeño, a continuación se muestran los diferentes modelos construidos y evaluados.

Modelo	Capa 1		Capa 2		Capa 3		Capa 4	Capa Salida	F1-Score
	Neuronas	Función	Neuronas	Función	Neuronas	Función	Neuronas	Función	Valor
Modelo 1	32	ReLU	N/A	N/A	N/A	N/A	N/A	Sigmoide	0.55
Modelo 2	64	ReLU	32	ReLU	N/A	N/A	N/A	Sigmoide	0.617
Modelo 3	128	ReLU	64	ReLU	32	ReLU	N/A	Sigmoide	0.675
Modelo 4	64	ReLU	64	ReLU	32	ReLU	16	Sigmoide	0.66
Modelo 5	128	ReLU	64	ReLU	64	ReLU	N/A	Sigmoide	0.671
Modelo 6	128	LeakyReLU	64	LeakyReLU	32	LeakyReLU	N/A	Sigmoide	0.676

Table 2: Combinaciones de capas, funciones de activación y F1-Score en diferentes modelos.

3.4.4 Regularización con Dropout

Un aspecto fundamental de los modelos presentados en la Tabla 2 es la implementación de un método de regularización conocido como Dropout. Esta técnica es ampliamente utilizada para prevenir sobreajuste (overfitting) en redes neuronales profundas.

Durante el entrenamiento, se apagan (desactivan) aleatoriamente ciertas neuronas en cada capa con una probabilidad específica, conocida como tasa de dropout (en este caso se usó 0.3). Esto implica que las conexiones hacia y desde las neuronas desactivadas no se utilizan durante ese paso de entrenamiento. En cada época, el subconjunto de neuronas desactivadas cambia, forzando al modelo a aprender patrones más generales y a no depender exclusivamente de ciertas rutas neuronales específicas.

En la fase de inferencia (es decir, durante las predicciones con datos de prueba), todas las neuronas están activas. Sin embargo, los pesos de las conexiones se escalan proporcionalmente para tener en cuenta la desactivación aleatoria que ocurrió durante el entrenamiento, lo que mantiene la consistencia entre las fases de entrenamiento e inferencia.

3.4.5 Otros parámetros

Los modelos presentados fueron entrenados utilizando 20, 30 y 50 épocas para evaluar su desempeño en diferentes etapas del entrenamiento. Además, todos ellos utilizaron un batch size de 32, un valor ampliamente utilizado en la literatura debido a su balance entre estabilidad del gradiente y eficiencia computacional.

Por otro lado, la **función de activación** utilizada en las capas ocultas de todos los modelos (excepto en el Modelo 6) corresponde a la *Rectified Linear Unit (ReLU)*. Esta función activa únicamente los valores positivos mediante la siguiente expresión:

$$f(x) = \max(0, x)$$

El uso de ReLU permite una eficiencia computacional significativa al simplificar los cálculos y evitar la saturación de los gradientes, lo que acelera el proceso de entrenamiento.

En el **Modelo 6**, se decidió utilizar la *Leaky ReLU*, una variante de ReLU, con el objetivo de probar su desempeño. Esta función permite que los valores negativos pasen con un pequeño gradiente, evitando así que las neuronas se apaguen por completo. La función Leaky ReLU se define como:

$$f(x) = \begin{cases} x & \text{si } x \geq 0 \\ \alpha x & \text{si } x < 0 \end{cases}$$

donde α es una pequeña constante, típicamente 0.01.

En todos los modelos, se utilizó la **función Sigmoide** en la capa de salida, ya que la tarea de predicción es de carácter binario (1 = pobre, 0 = no pobre). La función sigmoide convierte la salida en un valor entre 0 y 1, interpretado como la probabilidad de pertenecer a la clase "pobre":

$$f(x) = \frac{1}{1 + e^{-x}}$$

Este enfoque permite que la red neuronal asigne una probabilidad a cada predicción, lo que es especialmente útil en problemas de clasificación binaria como el que se aborda en este trabajo.

Finalmente vale la pena mencionar que la red fue entrenada con el fin de maximizar el *accuracy*. Aunque esto puede parecer contradictorio dado que el *F1* es quien se busca maximizar, una vez se realizó el proceso de remuestreo, se evidencio con múltiples pruebas que cuando se buscaba maximizar el *F1* score, se obtenían peores resultados que cuando se buscaba maximizar el *accuracy*.

A continuación, se muestra el resultado final de la red implementada con los componentes previamente mencionados.

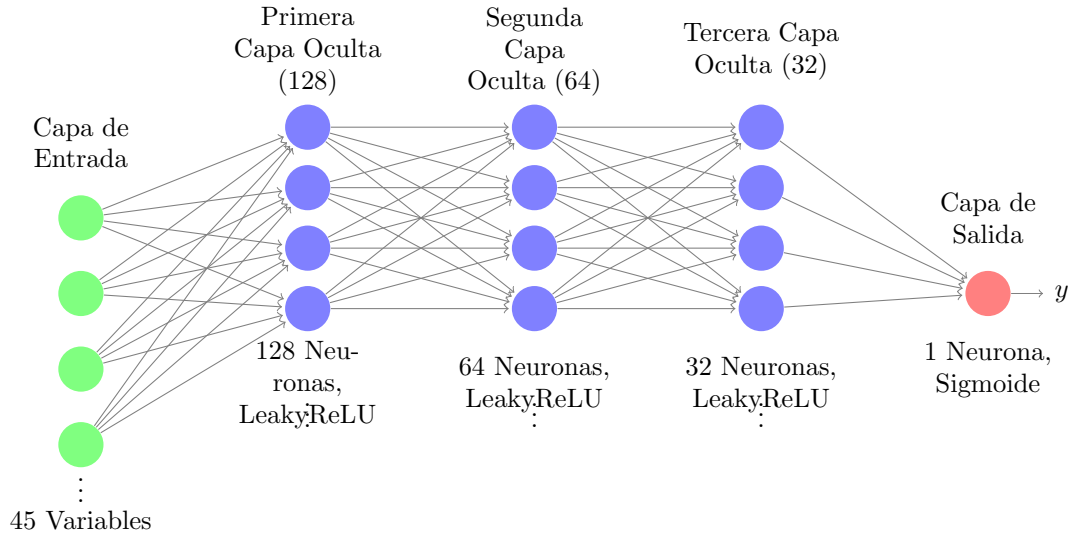


Figure 4: Esquema de la red neuronal con mejor desempeño (Modelo 6)

3.4.6 Resultados

El entrenamiento del modelo se realizó utilizando Python, junto con las bibliotecas Keras y Scikit-learn. El tiempo total de ejecución en entrenamiento fue de 14 minutos y 22 segundos.

Los experimentos computacionales se llevaron a cabo en un dispositivo denominado IIIND-PIRYBKR3. El sistema cuenta con un procesador Intel(R) Core(TM) i7-1265U de 12^a generación, con una frecuencia base de 1.80 GHz. La máquina está equipada con 16 GB de memoria RAM instalada (15.7 GB utilizables) y ejecuta un sistema operativo de 64 bits basado en la arquitectura x64.

A continuación se muestra el proceso de entrenamiento a lo largo de las 50 épocas utilizadas, para el conjunto de validación y entrenamiento.

La Figura 5 muestra el estancamiento ligero en el proceso de entrenamiento posterior a las 40 épocas en el conjunto de validación, por lo cual, con el fin de evitar también procesos de sobre-ajuste, se realiza parada temprana.

$$F1_{Kaggle} = 0.676 \quad (4)$$

A pesar del esfuerzo realizado en la construcción de variables para considerar interacciones y relaciones explícitas entre las mismas, las redes neuronales parecen haber identificado relaciones latentes adicionales que contribuyeron a mejorar su desempeño.

Si bien las redes neuronales son modelos con una alta complejidad y una mayor probabilidad de sobreajuste, los resultados sugieren que la regularización de los parámetros se realizó de manera adecuada, permitiendo un entrenamiento eficiente y evitando el sobreajuste del modelo.

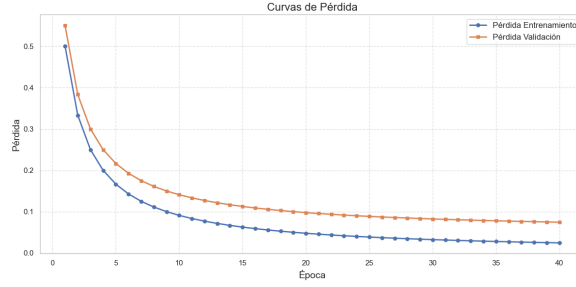


Figure 5: Gráficas de perdida en entrenamiento y validación

Vale la pena mencionar que el modelo que tuvo mayor proximidad al mejor desempeño en la competencia fue la regresión logística, el cual es el modelo más sencillo en términos de complejidad. Sin embargo, parece indicar que las variables seleccionadas capturaban de cierta forma la complejidad de la variable a predecir. Por otro lado, modelos como CARTS, Random Forest y XgBoost, mostraron un desempeño precario en evaluación. Aunque los métodos de arboles parecían ser grandes contrincantes contra un modelo basado en redes profundas, todo indica a que estos modelos tendieron a realizar procesos de sobreajuste ya que sus resultados en entrenamiento fueron muy buenos, pero en evaluación fueron bastante inferiores.

La tabla a continuación resume los resultados obtenidos en la evaluación en Kaggle:

Modelo	F1-Score	Orden
D.N.N	0.676	1
Regresión Logística	0.638	2
XGBoost	0.605	3
Random Forest	0.576	4
CARTS	0.469	5

Table 3: Comparación de modelos con sus respectivos F1-Scores y orden de desempeño.

La Tabla 3 destaca un resultado interesante: el modelo con mayor interpretabilidad (regresión logística) y el de menor interpretabilidad (D.N.N) fueron los que lograron el mejor desempeño. Este hecho sugiere que, a lo largo del trabajo, la selección y construcción de variables fue clave para mejorar la precisión del modelo. Esto indica que las variables seleccionadas y construidas capturan de manera adecuada la complejidad del problema abordado.

Por un lado, las redes neuronales aprendieron automáticamente las relaciones complejas entre las variables, posiblemente capturando patrones que la regresión logística no identificó. Sin embargo, la simplicidad de la regresión logística permitió al modelo mantener una baja varianza, es decir, una mayor estabilidad en sus predicciones.

Este resultado ilustra claramente el trade-off entre sesgo y varianza: aunque el modelo más complejo (D.N.N) tiene una mayor varianza, logró superarse mediante el uso adecuado de métodos de regularización. Por otro lado, la regresión logística, con menor complejidad y mayor sesgo, aprovechó al máximo la adecuada selección de variables, demostrando el poder predictivo que pueden tener los modelos simples cuando se trabaja cuidadosamente en la preparación de los datos.

4 Conclusiones

La predicción de la pobreza es una tarea compleja y costosa en diversas dimensiones, por lo que es fundamental encontrar modelos que permitan una detección rápida y eficiente. Alcanzar este objetivo puede facilitar la formulación de políticas públicas más efectivas y oportunas. Este trabajo exploró la posibilidad de predecir si un hogar es pobre o no, a partir de algunas de sus características y las de los individuos que lo integran. Para ello, se utilizaron métodos estadísticos y técnicas de machine learning con el propósito de identificar los predictores y modelos más adecuados para lograr una aproximación precisa a la realidad.

El enfoque se centró en modelos de clasificación como Random Forest, Árboles de decisión, XG-Boost, Regresión Logística y Redes Neuronales Profundas. Los mejores resultados en las evaluaciones fueron obtenidos por las redes neuronales y la regresión logística, modelos que fueron optimizados con el objetivo de manejar el trade-off entre sesgo y varianza, un reto común en machine learning.

El modelo más destacado en términos predictivos fue una red neuronal profunda con una arquitectura sencilla. Esta red recibió datos previamente escalados y remuestreados para corregir el desbalanceo entre clases. El modelo se diseñó para capturar relaciones complejas entre las variables, mientras que se aplicaron técnicas de regularización, como dropout, para evitar el sobreajuste, un problema habitual en redes neuronales.

Dada la complejidad del problema de predicción, es razonable esperar que un modelo basado en redes neuronales ofrezca el mejor rendimiento, ya que tiene la capacidad de capturar comportamientos e interacciones complejas entre las variables. Sin embargo, es importante destacar que el proceso de construcción y selección de las variables predictoras fue, en este caso, el factor que más contribuyó a mejorar el desempeño del modelo. Esto sugiere que la combinación de modelos complejos con una selección cuidadosa y fundamentada de los predictores puede conducir a un rendimiento superior. En este contexto, variables como el nivel de hacinamiento, el arriendo estimado del hogar y el grado máximo de escolaridad en el hogar resultaron esenciales para proporcionar información relevante sobre la situación del hogar.

En trabajos futuros se podría explorar la creación de modelos de regresión o clasificación que faciliten la obtención de predictores más elaborados y completos, como el estrato socioeconómico del hogar. Además, dado que las redes neuronales profundas muestran su máximo potencial en entornos con alta dimensionalidad y grandes volúmenes de datos, la incorporación de una mayor cantidad de información podría permitir al modelo aumentar su complejidad sin incurrir en sobreajuste. Esto contribuiría a mejorar la precisión y confiabilidad de las predicciones, proporcionando un mayor grado de certeza en los resultados obtenidos.

5 Apendice

5.1 Archivos

1. **ddi-documentation-spanish-608 (PDF)**: Documento que describe las variables y algunas precisiones sobre la forma de muestreo.
2. **train_hogares.csv**: Conjunto de entrenamiento que contiene observaciones de hogares con múltiples variables descriptivas.
3. **train_personas.csv**: Conjunto de entrenamiento que contiene información sobre las personas que habitan en los hogares muestreados.
4. **test_hogares.csv**: Conjunto de evaluación que contiene observaciones de hogares con múltiples variables descriptivas (Posee menos variables que el conjunto de entrenamiento).
5. **test_personas.csv**: Conjunto de evaluación que contiene información sobre las personas que habitan en los hogares muestreados (Posee menos variables que el conjunto de entrenamiento).
6. **sample_submission.csv**: Contiene un ejemplo de la estructura que debe poseer un archivo de evaluación para medir el rendimiento del modelo en competencia.

5.2 Definición de variables

1. **Hacinación**: Se definió una medida de hacinación h_i para cada hogar i a partir del número de personas que habitan en el hogar y el número de cuartos que son usados en el hogar para dormir, esto es:

$$h_i = \frac{n_i^{\text{personas}}}{m_i^{\text{cuartos}}}$$

Donde se espera que en hogares pobres, el grado de hacinamiento sea mayor.

2. **Escolaridad media:** Se definió un promedio de escolaridad $\bar{e}_i$ para cada hogar i como la suma ponderada del grado de escolaridad más alto alcanzado por cada miembro j del hogar. Esto es:

$$\bar{e}_i = \frac{\sum_j g_j^{\text{escolaridad}}}{n_i^{\text{personas}}}$$

Donde se espera que en hogares pobres, el grado de escolaridad medio sea menor.

3. **Subsidios:** Teniendo en cuenta que existen múltiples subsidios dentro de la base de datos, se utilizaron los que pueden tener un mayor indicio de precariedad económica por parte del hogar.

- **Subsidio familiar:** Se realiza un conteo de la cantidad de personas en el hogar que reciben subsidio familiar en el hogar. En caso de que exista este tipo de subsidio, se espera que el hogar sea pobre.
- **Subsidio educativo:** Se realiza un conteo de la cantidad de personas en el hogar que reciben subsidio educativo en el hogar. En caso de que exista este tipo de subsidio, se espera que el hogar sea pobre.
- **Subsidio de alimentación:** Se realiza un conteo de la cantidad de personas en el hogar que reciben subsidio de alimentación en el hogar. En caso de que exista este tipo de subsidio, se espera que el hogar sea pobre.

4. **Cantidad de personas ocupadas en el hogar:** Entendiendo la ocupación como un parámetro binario asignado a cada persona como: $o_j = 1$ si la persona j esta ocupada, se calculo el número de personas ocupadas en el hogar O_i como un indicativo de desempleo, esto es:

$$O_i = \sum_j o_j$$

En este caso, se espera que el número de personas ocupadas en un hogar pobre sea bajo, indicando posible desempleo dentro del hogar.

5. **Número de personas cotizando pensión en el hogar:** Entendiendo la cotización en un fondo pensiones como un parámetro binario asignado a cada persona como: $c_j = 1$ si la persona j cotiza, se calculo el número de personas que se encuentran realizando cotizaciones en algún fondo de pensiones dentro del hogar C_i como un indicativo de formalidad laboral, esto es:

$$C_i = \sum_j c_j$$

En este caso, se espera que el número de personas que cotizan en un hogar pobre sea bajo, indicando una posible informalidad laboral dentro del hogar.

6. **Título del hogar:** Para cada hogar, se determinó en base a la encuesta que tipo de vivienda es dentro de las siguientes opciones: (a. Propia, totalmente pagada b. Propia, la están pagando c. En arriendo o subarriendo d. En usufructo e. Posesión sin título (ocupante f. Otra). Esta variable fue construida como una **variable categórica**.
7. **Arriendo estimado del hogar:** Se tuvo en cuenta la estimación del arriendo del hogar por parte de los propietarios/ habitantes. Dado que en múltiples ocasiones este valor no era reportado dentro de la base de datos, en caso de que el valor fuera nulo se utilizo la media del resto de observaciones como estimador de estos valores de ser necesario.
8. **Número de personas en el hogar:** Adicional a la medida de hacinamiento previamente definida, se tuvo en cuenta le número de personas n_i que habita en el hogar i como otra medida de hacinamiento que puede brindar información sobre precariedad del hogar.

5.3 Listado de variables utilizadas en el modelo

- P6210
- P6020
- P6040
- P6050
- P6090
- Fex_c
- Dominio_BARRANQUILLA
- Dominio_BUCARAMANGA
- Dominio_CALI
- Dominio_CARTAGENA
- Dominio_CUCUTA
- Dominio_FLORENCIA
- Dominio_IBAGUE
- Dominio_MANIZALES
- Dominio_MEDELLIN
- Dominio_MONTERIA
- Dominio_NEIVA
- Dominio_PASTO
- Dominio_PEREIRA
- Dominio_POPAYAN
- Dominio_QUIBDO
- Dominio_RESTO URBANO
- Dominio_RIOHACHA
- Dominio_RURAL
- Dominio_SANTA MARTA
- Dominio_SINCELEJO
- Dominio_TUNJA
- Dominio_VALLEDUPAR
- Dominio_VILLAVICENCIO
- Npersug
- Cotizan_p
- Nper
- Hasinación
- mean_escolaridad
- mean_escolaridad_max
- Subsidios
- Ocupados
- P5090_2
- P5090_3
- P5090_4
- P5090_5
- P5090_6
- P5130
- Subsidio_a
- Subsidio_e

5.4 Calculo del Score $F1$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

Donde se define cada termino a partir de la matriz de confusión como:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

5.4.1 SMOTE

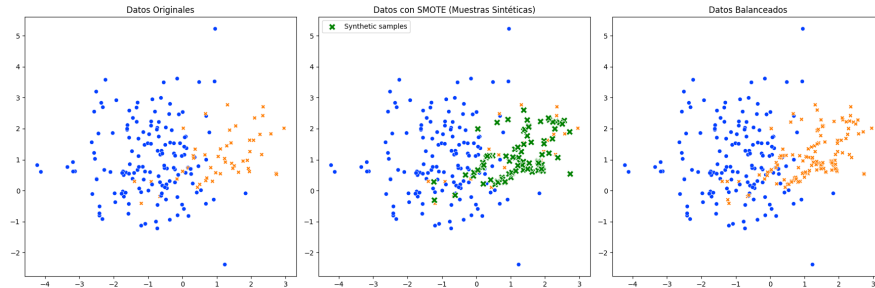


Figure 6: Ejemplificación del método SMOTE

References

- [1] Utilities of artificial intelligence in poverty prediction: A review. *MDPI*. Accedido: 20 de octubre de 2024.

- [2] Poverty rate prediction using multi-modal survey and earth observation data. *arXiv preprint arXiv:2307.11921*, 2023. Accedido: 20 de octubre de 2024.
- [3] Departamento Administrativo Nacional de Estadística (DANE). Boletín Técnico: Pobreza Multi-dimensional en Colombia, 2020, 2020. Accedido: 20 de octubre de 2024.
- [4] Ji Yoon Kim. Using machine learning to predict poverty status in costa rican households. *arXiv preprint arXiv:2111.13319*, 2021. Accedido: 20 de octubre de 2024.
- [5] World Bank. Machine learning and the future of poverty prediction, 2024. Accedido: 20 de octubre de 2024.